



Retrieval Augmented Generation options and architectures on AWS

AWS Prescriptive Guidance



AWS Prescriptive Guidance: Retrieval Augmented Generation options and architectures on AWS

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Introduction	1
Intended audience	1
Objectives	1
Generative AI options	3
Understanding RAG	4
Components of production-level RAG systems	6
Comparing RAG and fine-tuning	7
Use cases for RAG	10
Fully managed RAG options	11
Knowledge bases for Amazon Bedrock	11
Data sources for knowledge bases	13
Vector databases for knowledge bases	15
Amazon Q Business	15
Key features	15
End-user customization	17
Amazon SageMaker AI Canvas	17
Custom RAG architectures	20
Retrievers	20
Amazon Kendra	21
Amazon OpenSearch Service	22
Amazon Aurora PostgreSQL and pgvector	23
Amazon Neptune Analytics	24
Amazon MemoryDB	24
Amazon DocumentDB	26
Pinecone	28
MongoDB Atlas	29
Weaviate	30
Generators	30
Amazon Bedrock	31
SageMaker AI JumpStart	31
Choosing a RAG option	32
Conclusion	34
Document history	35

Retrieval Augmented Generation options and architectures on AWS

Mithil Shah, Natacha Fort, and Rajeev Muralidhar, Amazon Web Services

Generative AI refers to a subset of AI models that can create new content and artifacts, such as images, videos, text, and audio, from a simple text prompt. Generative AI models are trained on vast amounts of data that encompasses a wide range of subjects and tasks. This enables them to demonstrate remarkable versatility in performing various tasks, even those for which they have not been explicitly trained. Due to a single model's ability to perform multiple tasks, these models are often referred to as *foundation models* (FMs).

One of the notable applications of generative AI models is their proficiency in answering questions. However, there are specific challenges that arise when these models are used to answer questions based on custom documents. Custom documents can include proprietary information, internal websites, internal documentation, Confluence pages, SharePoint pages, and others. One option is to use *Retrieval Augmented Generation* (RAG). With RAG, the foundation model references an authoritative data source that is outside of its training data sources (such as your custom documents) before generating a response.

This guide describes the distinct generative AI options that are available for answering questions from custom documentation, including Retrieval Augmented Generation (RAG) systems. It also provides an overview of building RAG systems on Amazon Web Services (AWS). By reviewing the RAG options and architectures, you can choose between fully managed services on AWS and custom RAG architectures.

Intended audience

The intended audience for this guide is generative AI architects and managers who want to build a RAG solution, to review the available architectures, and to understand the benefits and drawbacks of each option.

Objectives

This guide helps you do the following:

- Understand the generative AI options available for answering questions from custom documents

- Review the architecture options for RAG systems on AWS
- Understand the advantages and disadvantages of each RAG option
- Choose a RAG architecture for your AWS environment

Generative AI options for querying custom documents

Organizations often have various sources of structured and unstructured data. This guide focuses on how you can use generative AI to answer questions from unstructured data.

Unstructured data in your organization can come from various sources. These might be PDFs, text files, internal wikis, technical documents, public facing websites, knowledge bases, or others. If you want a foundation model that can answer questions about unstructured data, the following options are available:

- Train a new foundation model by using your custom documents and other training data
- Fine-tune an existing foundation model by using data from your custom documents
- Use in-context learning to pass a document to the foundation model when you ask a question
- Use a Retrieval Augmented Generation (RAG) approach

Training a new foundation model from scratch that includes your custom data is an ambitious undertaking. A few companies have done it successfully, such as Bloomberg with their [BloombergGPT](#) model. Another example is the multimodal [EXAONE](#) model by LG AI Research, which was trained by using 600 billion pieces of artwork and 250 million high-resolution images, accompanied with text. According to [The Cost of AI: Should You Build or Buy Your Foundation Model](#) (LinkedIn), a model similar to Meta Llama 2 costs around USD \$4.8 million to train. There are two primary prerequisites for training a model from scratch: access to resources (financial, technical, time) and a clear return on investment. If this does not seem the right fit, then the next option is to fine-tune an existing foundation model.

Fine-tuning an existing model involves taking a model, such as an Amazon Titan, Mistral, or Llama model, and then adapting the model to your custom data. There are various techniques for fine-tuning, most of which involve modifying only a few parameters instead of modifying all of the parameters in the model. This is called *parameter-efficient fine-tuning*. There are two primary methods for fine-tuning:

- *Supervised fine-tuning* uses labeled data and helps you train the model for a new kind of task. For example, if you wanted to generate a report based on a PDF form, then you might have to teach the model how to do that by providing enough examples.

- *Unsupervised fine-tuning* is task-agnostic and adapts the foundation model to your own data. It trains the model to understand the context of your documents. The fine-tuned model then creates content, such as a report, by using a style that is more custom your organization.

However, fine-tuning may not be ideal for question-answer use cases. For more information, see [Comparing RAG and fine-tuning](#) in this guide.

When you ask a question, you can pass a document the foundation model and use the model's in-context learning to return answers from the document. This option is suitable for ad-hoc querying of a single document. However, this solution doesn't work well for querying multiple documents or for querying systems and applications, such as Microsoft SharePoint or Atlassian Confluence.

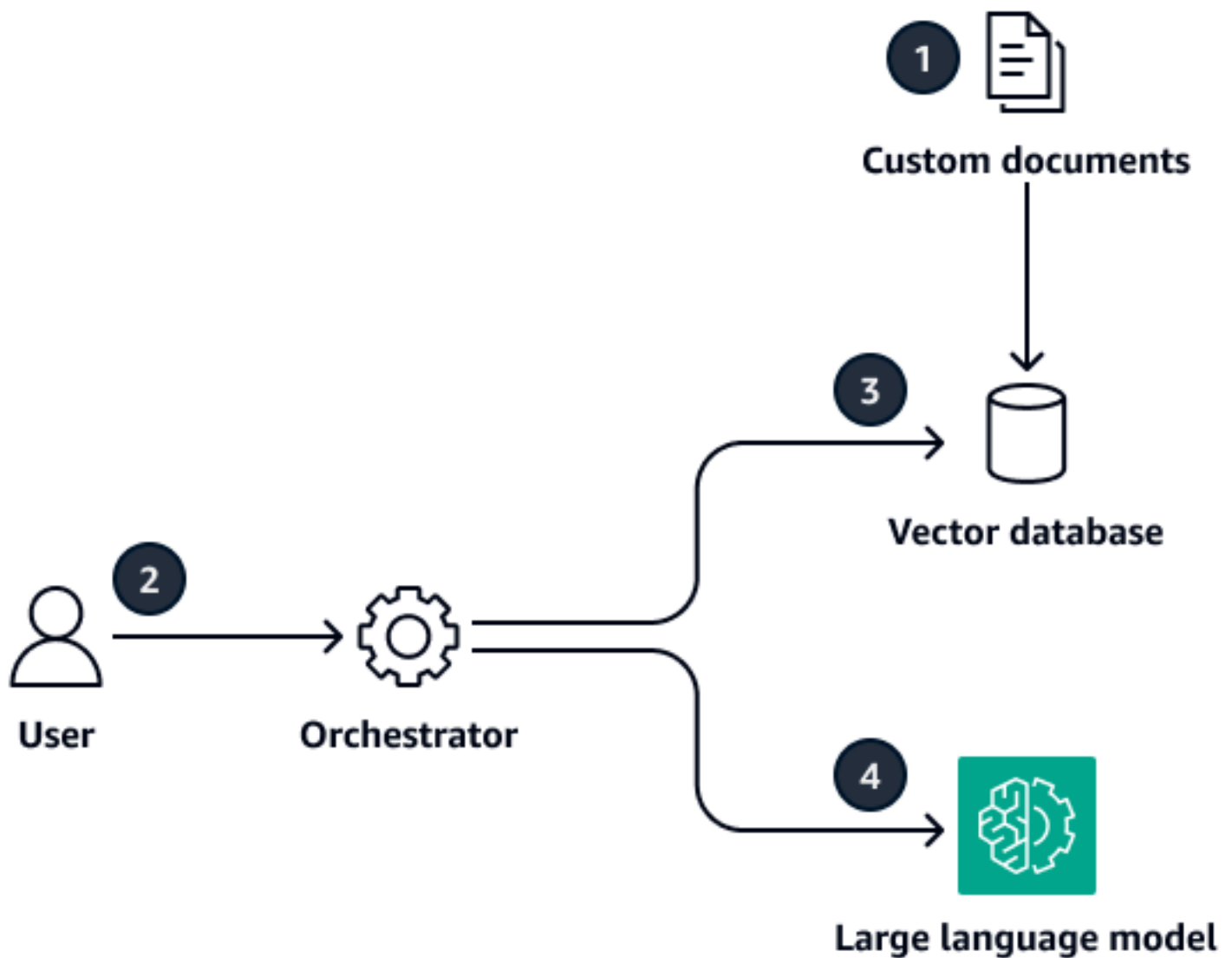
The final option is to use RAG. With RAG, the foundation model references your custom documents before generating a response. RAG extends the model's capabilities to your organization's internal knowledge base, all without the need to retrain the model. It is a cost-effective approach to improving the model output so that it remains relevant, accurate, and useful in various contexts.

Topics in this section:

- [Understanding RAG](#)
- [Comparing RAG and fine-tuning](#)
- [Use cases for RAG](#)

Understanding Retrieval Augmented Generation

Retrieval Augmented Generation (RAG) is a technique used to augment a large language model (LLM) with external data, such as a company's internal documents. This provides the model with the context it needs to produce accurate and useful output for your specific use case. RAG is a pragmatic and effective approach to using LLMs in an enterprise. The following diagram shows a high-level overview of how a RAG approach works.



Broadly speaking, the RAG process is four steps. The first step is done once, and the other three steps are done as many times as needed:

1. You create embeddings to ingest the internal documents into a vector database. *Embeddings* are numeric representations of text in the documents that capture the semantic or contextual meaning of the data. A *vector database* is essentially a database of these embeddings, and it is sometimes called a *vector store* or *vector index*. This step requires data cleaning, formatting, and chunking, but this is a one-time, upfront activity.
2. A human submits a query in natural language.
3. An orchestrator performs a similarity search in the vector database and retrieves the relevant data. The orchestrator adds the retrieved data (also known as *context*) to the prompt that contains the query.

4. The orchestrator sends the query and the context to the LLM. The LLM generates a response to the query by using the additional context.

From a user's perspective, RAG looks like interacting with any LLM. However, the system knows much more about the content in question and provides answers that are fine-tuned to the organization's knowledge base.

For more information about how a RAG approach works, see [What is RAG](#) on the AWS website.

Components of production-level RAG systems

Building a production-level RAG system requires thinking through several different aspects of the RAG workflow. Conceptually, a production-level RAG workflow requires the following capabilities and components, regardless of the specific implementation:

- **Connectors** — These connect different enterprise data sources with the vector database. Examples of structured data sources include transactional and analytical databases. Examples of unstructured data sources include object stores, code bases, and software as a service (SaaS) platforms. Each data source might require different connectivity patterns, licenses, and configurations.
- **Data processing** – Data comes in many shapes and forms, such as PDFs, scanned images, documents, presentations, and Microsoft SharePoint files. You must use data processing techniques to extract, process, and prepare the data for indexing.
- **Embeddings** — To perform a relevancy search, you must convert your documents and user queries into a compatible format. By using embedding language models, you convert the documents into numerical representation. These are essentially inputs for the underlying foundation model.
- **Vector database** — The vector database is an index of the embeddings, the associated text, and metadata. The index is optimized for search and retrieval.
- **Retriever** — For the user query, the retriever fetches the relevant context from the vector database and ranks the responses based on business requirements.
- **Foundation model** — The foundation model for a RAG system is typically an LLM. By processing the context and the prompt, the foundation model generates and formats a response for the user.
- **Guardrails** — Guardrails are designed to make sure that the query, prompt, retrieved context, and LLM response are accurate, responsible, ethical, and free of hallucinations and bias.

- **Orchestrator** — The orchestrator is responsible for scheduling and managing the end-to-end workflow.
- **User experience** — Typically, the user interacts with a conversational chat interface that has rich features, including displaying chat history and collecting user feedback about responses.
- **Identity and user management** – It is critical to control user access to the application at fine granularity. In the AWS Cloud, policies, roles, and permissions are typically managed through [AWS Identity and Access Management \(IAM\)](#).

Clearly, there is significant amount of work to plan, develop, release, and manage a RAG system. [Fully managed services](#), such as Amazon Bedrock or Amazon Q Business, can help you manage some of the undifferentiated heavy lifting. However, [custom RAG architectures](#) can provide more control over the components, such as the retriever or the vector database.

Comparing Retrieval Augmented Generation and fine-tuning

The following table describes the advantages and disadvantages of the fine-tuning and RAG-based approaches.

Approach	Advantages	Disadvantages
Fine-tuning	• If a fine-tuned model is trained using the unsupervised approach, then it is able to create content that more closely matches your organization's style. • A fine-tuned model that is trained on proprietary or regulatory data can help your organization follow in-house or industry-specific data and compliance standards.	• Fine-tuning can take a few hours to days, depending on the size of the model. Therefore, it not be a good solution if your custom documents change frequently. • Fine-tuning requires an understanding of techniques, such as low-rank adaptation (LoRA) and parameter-efficient fine-tuning (PEFT). Fine-tuning might require a data scientist.

- Fine-tuning might not be available for all models.
- Fine-tuned models do not provide a reference to the source in their responses.
- There can be an increased risk of hallucination when using a fine-tuned model to answer questions.

RAG

- RAG allows you to build a question-answering system for your custom documents without fine-tuning.
- RAG does not work well when summarizing information from entire documents.
- RAG can incorporate the latest documents in a few minutes.
- AWS offers fully managed RAG solutions. Therefore, no data scientist or specialized knowledge of machine learning is required.
- In its response, a RAG model provides a reference to the information source.
- Because RAG uses the context from the vector search as the basis of its generated answer, there is a reduced risk of hallucination.

If you need to build a question-answering solution that references your custom documents, then we recommend that you start from a RAG-based approach. Use fine-tuning if you need the model to perform additional tasks, such as summarization.

You can combine the fine-tuning and RAG approaches in a single model. In the case, the RAG architecture does not change, but the LLM that generates the answer is also fine-tuned with the custom documents. This combines the best of both worlds, and it might be an optimum solution

for your use case. For more information about how to combine supervised fine-tuning with RAG, see the [RAFT: Adapting Language Model to Domain Specific RAG](#) research from the University of California, Berkeley.

Use cases for Retrieval Augmented Generation

The following are common use cases for using a RAG approach:

- **Search engines** – RAG-enabled search engines can provide more accurate and up-to-date featured snippets in their search results.
- **Question-answering systems** – RAG can improve the quality of responses in question-answering systems. The retrieval-based model uses similarity search to find relevant passages or documents that contain the answer. Then, it generates a concise and relevant response based on that information.
- **Retail or e-commerce** – RAG can enhance the user experience in e-commerce by providing more relevant and personalized product recommendations. By retrieving and incorporating information about user preferences and product details, RAG can generate more accurate and helpful recommendations for customers.
- **Industrial or manufacturing** – In manufacturing, RAG helps you quickly access critical information, such as factory plant operations. It can also help with decision-making processes, troubleshooting, and organizational innovation. For manufacturers who operate within stringent regulatory frameworks, RAG can swiftly retrieve updated regulations and compliance standards from internal and external sources, such as from industry standards or regulatory agencies.
- **Healthcare** – RAG has potential in the healthcare industry, where access to accurate and timely information is crucial. By retrieving and incorporating relevant medical knowledge from external sources, RAG can provide more accurate and context-aware responses in healthcare applications. Such applications augment the information accessible by a human clinician, who ultimately makes the call and not the model.
- **Legal** – RAG can be applied powerfully in legal scenarios, such as mergers and acquisitions, where complex legal documents provide context for queries. This can help legal professionals rapidly navigate complex regulatory issues.

Fully managed Retrieval Augmented Generation options on AWS

To manage Retrieval Augmented Generation (RAG) workflows on AWS, you can use custom RAG pipelines or use some of the fully managed services capabilities that AWS offers. Because they include many of the core components of a RAG-based system, fully managed services can help you manage some of the undifferentiated heavy lifting. However, these services provide less opportunity for customization.

The fully managed AWS services use connectors to ingest data from external data sources, such as websites, Atlassian Confluence, or Microsoft SharePoint. The supported data sources vary by AWS service.

This section explores the following fully managed options for building RAG workflows on AWS:

- [Knowledge bases for Amazon Bedrock](#)
- [Amazon Q Business](#)
- [Amazon SageMaker AI Canvas](#)

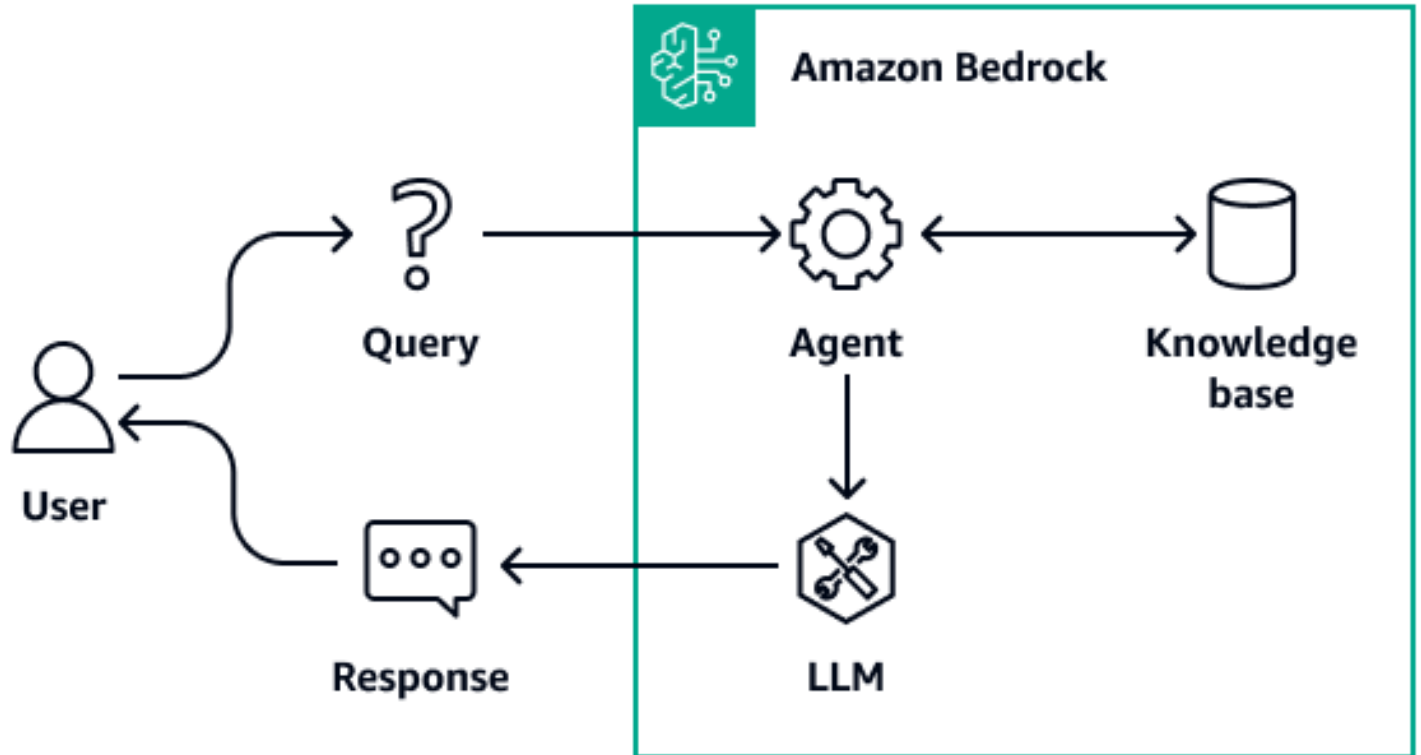
For more information about how to choose between these options, see [Choosing a RAG option](#) in this guide.

Knowledge bases for Amazon Bedrock

[Amazon Bedrock](#) is a fully managed service that makes high-performing foundation models (FMs) from leading AI startups and Amazon available for your use through a unified API. [Knowledge bases](#) is an Amazon Bedrock capability that helps you implement the entire RAG workflow, from ingestion to retrieval and prompt augmentation. There is no need to build custom integrations to data sources or to manage data flows. Session context management is built in so that your generative AI application can readily support multi-turn conversations.

After you specify the location of your data, knowledge bases for Amazon Bedrock internally fetches the documents, chunks them into blocks of text, converts the text to embeddings, and then stores the embeddings in your choice of vector database. Amazon Bedrock manages and updates the embeddings, keeping the vector database in sync with the data. For more information about how knowledge bases work, see [How Amazon Bedrock knowledge bases work](#).

If you add knowledge bases to an Amazon Bedrock agent, the agent identifies the appropriate knowledge base based on the user input. The agent retrieves the relevant information and adds the information to the input prompt. The updated prompt provides the model with more context information to generate a response. To improve transparency and minimize hallucinations, the information retrieved from the knowledge base is traceable to its source.



Amazon Bedrock supports the following two APIs for RAG:

- [RetrieveAndGenerate](#) – You can use this API to query your knowledge base and generate responses from the information it retrieves. Internally, Amazon Bedrock converts the queries into embeddings, queries the knowledge base, augments the prompt with the search results as context information, and returns the LLM-generated response. Amazon Bedrock also manages the short-term memory of the conversation to provide more contextual results.
- [Retrieve](#) – You can use this API to query your knowledge base with information retrieved directly from the knowledge base. You can use the information returned from this API to process the retrieved text, evaluate their relevance, or develop a separate workflow for response generation. Internally, Amazon Bedrock converts the queries into embeddings, searches the knowledge base, and returns the relevant results. You can build additional workflows on top of the search results. For example, you can use the [LangChain](#) `AmazonKnowledgeBasesRetriever` plugin to integrate RAG workflows into generative AI applications.

For sample architectural patterns and step-by-step instructions for using the APIs, see [Knowledge Bases now delivers fully managed RAG experience in Amazon Bedrock](#) (AWS blog post). For more information about how to use the RetrieveAndGenerate API to build a RAG workflow for an intelligent chat-based application, see [Build a contextual chatbot application using Amazon Bedrock Knowledge Bases](#) (AWS blog post).

Data sources for knowledge bases

You can connect your proprietary data to a knowledge base. After you've configured a data source connector, you can sync or keep your data up to date with your knowledge base and make your data available for querying. Amazon Bedrock knowledge bases support connections to the following data sources:

- [Amazon Simple Storage Service \(Amazon S3\)](#) – You can connect an Amazon S3 bucket to an Amazon Bedrock knowledge base by using either the console or the API. The knowledge base ingests and indexes the files in the bucket. This type of data source supports the following features:
 - **Document metadata fields** – You can include a separate file to specify the metadata for the files in the Amazon S3 bucket. You can then use these metadata fields to filter and improve the relevancy of responses.
 - **Inclusion or exclusion filters** – You can include or exclude certain content when crawling.
 - **Incremental syncing** – The content changes are tracked, and only content that has changed since the last sync is crawled.
- [Confluence](#) – You can connect an Atlassian Confluence instance to an Amazon Bedrock knowledge base by using the console or the API. This type of data source supports the following features:
 - **Auto detection of main document fields** – The metadata fields are automatically detected and crawled. You can use these fields for filtering.
 - **Inclusion or exclusion content filters** – You can include or exclude certain content by using a prefix or a regular expression pattern on the space, page title, blog title, comment, attachment name, or extension.
 - **Incremental syncing** – The content changes are tracked, and only content that has changed since the last sync is crawled.
 - **OAuth 2.0 authentication, authentication with Confluence API token** – The authentication credentials are stored in AWS Secrets Manager.

- [Microsoft SharePoint](#) – You can connect a SharePoint instance to a knowledge base by using either the console or the API. This type of data source supports the following features:
 - **Auto detection of main document fields** – The metadata fields are automatically detected and crawled. You can use these fields for filtering.
 - **Inclusion or exclusion content filters** – You can include or exclude certain content by using a prefix or a regular expression pattern on the main page title, event name, and file name (including its extension).
 - **Incremental syncing** – The content changes are tracked, and only content that has changed since the last sync is crawled.
 - **OAuth 2.0 authentication** – The authentication credentials are stored in AWS Secrets Manager.
- [Salesforce](#) – You can connect a Salesforce instance to a knowledge base by using either the console or the API. This type of data source supports the following features:
 - **Auto detection of main document fields** – The metadata fields are automatically detected and crawled. You can use these fields for filtering.
 - **Inclusion or exclusion content filters** – You can include or exclude certain content by using a prefix or a regular expression pattern. For a list of content types that you can apply filters to, see *Inclusion/exclusion filters* in the [Amazon Bedrock documentation](#).
 - **Incremental syncing** – The content changes are tracked, and only content that has changed since the last sync is crawled.
 - **OAuth 2.0 authentication** – The authentication credentials are stored in AWS Secrets Manager.
- [Web Crawler](#) – An Amazon Bedrock Web Crawler connects to and crawls the URLs that you provide. The following features are supported:
 - Select multiple URLs to crawl
 - Respect standard robots.txt directives, such as Allow and Disallow
 - Exclude URLs that match a pattern
 - Limit the rate of crawling
 - In Amazon CloudWatch, view the status of each URL crawled

For more information about the data sources that you can connect to your Amazon Bedrock knowledge base, see [Create a data source connector for your knowledge base](#).

Vector databases for knowledge bases

When you set up a connection between the knowledge base and the data source, you must configure a vector database, also known as a *vector store*. A vector database is where Amazon Bedrock stores, updates, and manages the embeddings that represent your data. Each data source supports different types of vector database. To determine which vector database are available for your data source, see the [data source types](#).

If you prefer for Amazon Bedrock to automatically create a vector database in Amazon OpenSearch Serverless for you, you can choose this option when you create the knowledge base. However, you can also choose to set up your own vector database. If you set up your own vector database, see [Prerequisites for your own vector store for a knowledge base](#). Each type of vector database has its own prerequisites.

Depending on your data source type, Amazon Bedrock knowledge bases support the following vector databases:

- [Amazon OpenSearch Serverless](#)
- [Amazon Aurora PostgreSQL-Compatible Edition](#)
- [Pinecone](#) (Pinecone documentation)
- [Redis Enterprise Cloud](#) (Redis documentation)
- [MongoDB Atlas](#) (MongoDB documentation)

Amazon Q Business

[Amazon Q Business](#) is a fully managed, generative-AI powered assistant that you can configure to answer questions, provide summaries, generate content, and complete tasks based on your enterprise data. It allows end users to receive immediate, permissions-aware responses from enterprise data sources with citations.

Key features

The following capabilities of Amazon Q Business can help you build a production-grade RAG-based generative AI application:

- **Built-in connectors** – Amazon Q Business supports more than 40 types of connectors, such as connectors for Adobe Experience Manager (AEM), Salesforce, Jira, and Microsoft SharePoint. For a complete list, see [Supported connectors](#). If you need a connector that is not supported, you can

use [Amazon AppFlow](#) to pull data from your data source into Amazon Simple Storage Service (Amazon S3) and then connect Amazon Q Business to the Amazon S3 bucket. For a complete list of data sources that Amazon AppFlow supports, see [Supported applications](#).

- **Built-in indexing pipelines** – Amazon Q Business provides a built-in pipeline for indexing data in a vector database. You can use an AWS Lambda function to add preprocessing logic for your indexing pipeline.
- **Index options** – You can create and provision a native index in Amazon Q Business, and you use an Amazon Q Business retriever to pull data from that index. Alternatively, you can use a preconfigured Amazon Kendra index as a retriever. For more information, see [Creating a retriever for an Amazon Q Business application](#).
- **Foundation models** – Amazon Q Business uses the foundation models that are supported in Amazon Bedrock. For a complete list, see [Supported foundation models in Amazon Bedrock](#).
- **Plugins** – Amazon Q Business provides the capability to use plugins to integrate with target systems, such as an automated way to summarize ticket information and ticket creation in Jira. Once configured, plugins can support read and write actions that can help you boost end user productivity. Amazon Q Business supports two types of plugins: [built-in plugins](#) and [custom plugins](#).
- **Guardrails** – Amazon Q Business supports global controls and topic-level controls. For example, these controls can detect personally identifiable information (PII), abuse, or sensitive information in prompts. For more information, see [Admin controls and guardrails in Amazon Q Business](#).
- **Identity management** – With Amazon Q Business, you can manage users and their access to the RAG-based generative AI application. For more information, see [Identity and access management for Amazon Q Business](#). Also, Amazon Q Business connectors index access control list (ACL) information that's attached to a document along with the document itself. Then, Amazon Q Business stores the ACL information it indexes in the Amazon Q Business User Store to create user and group mappings and filter chat responses based on the end user's access to documents. For more information, see [Data source connector concepts](#).
- **Document enrichment** – The document enrichment feature helps you control both **what** documents and document attributes are ingested into your index and also **how** they are ingested. This can be accomplished through two approaches:
 - **Configure basic operations** – Use basic operations to add, update, or delete document attributes from your data. For example, you can scrub PII data by choosing to delete any document attributes related to PII.
 - **Configure Lambda functions** – Use a preconfigured Lambda function to perform more customized, advanced document attribute manipulation logic to your data. For example,

your enterprise data might be stored as scanned images. In that case, you can use a Lambda function to run optical character recognition (OCR) on the scanned documents to extract text from them. Then, each scanned document is treated as a text document during ingestion. Finally, during chat, Amazon Q will factor the textual data extracted from the scanned documents when it generates responses.

When you implement your solution, you can choose to combine both document enrichment approaches. You can use basic operations to do a first parse of your data and then use a Lambda function for more complex operations. For more information, see [Document enrichment in Amazon Q Business](#).

- **Integration** – After you create your Amazon Q Business application, you can integrate it into other applications, such as Slack or Microsoft Teams. For example, see [Deploy a Slack gateway for Amazon Q Business](#) and [Deploy a Microsoft Teams gateway for Amazon Q Business](#) (AWS blog posts).

End-user customization

Amazon Q Business supports uploading documents that might not be stored in your organization's data sources and index. Uploaded documents are not stored. They are available for use only for the conversation in which the documents are uploaded. Amazon Q Business supports specific document types for upload. For more information, see [Upload files and chat in Amazon Q Business](#).

Amazon Q Business includes a [filtering by document attribute](#) feature. Both administrators and end users can use this feature. Administrators can customize and control chat responses for end users by using attributes. For example, if data source type is an attribute attached to your documents, you can specify that chat responses be generated only from a specific data source. Or, you can allow end users to restrict the scope of chat responses by using the attribute filters that you have selected.

End users can create lightweight, purpose-built [Amazon Q Apps](#) within your broader Amazon Q Business application environment. Amazon Q apps allow task automation for a specific domain, such as a purpose-built app for marketing team.

Amazon SageMaker AI Canvas

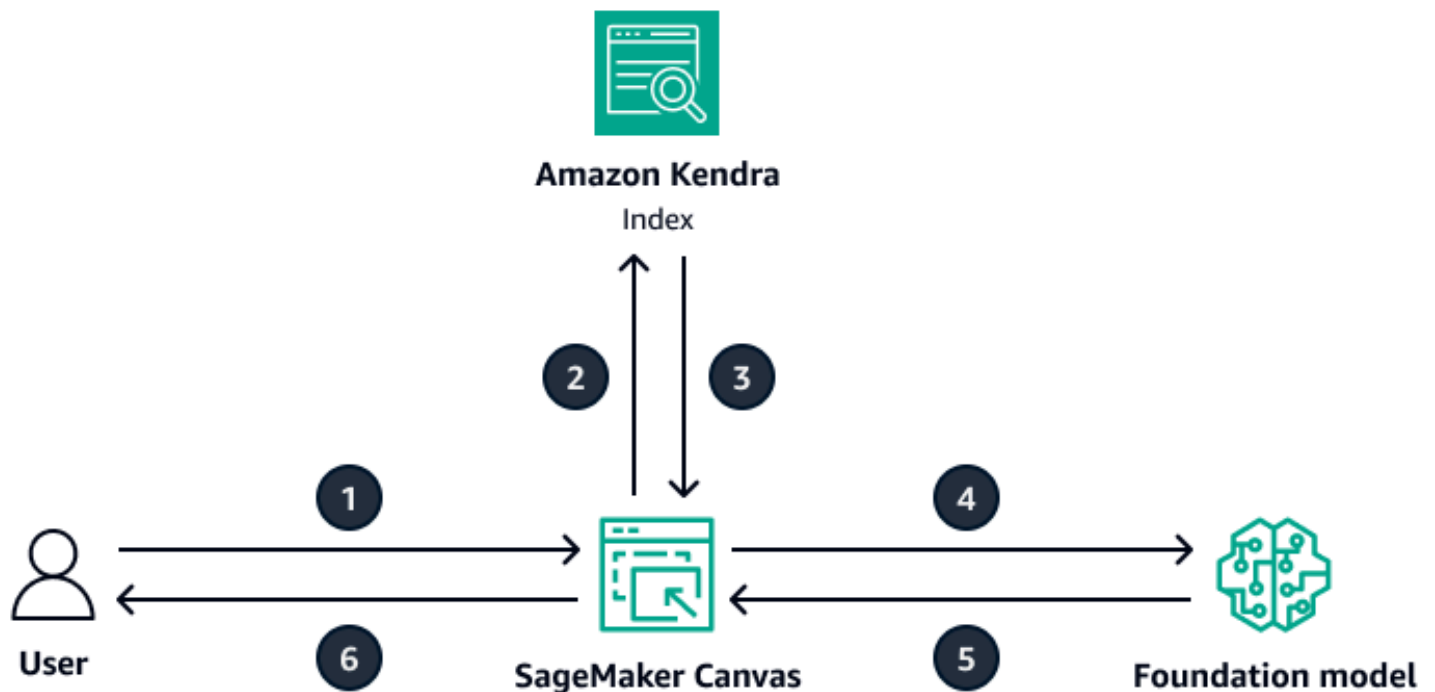
[Amazon SageMaker AI Canvas](#) helps you use machine learning to generate predictions without needing to write any code. It provides a no-code visual interface that empowers you to prepare

data, build, and deploy ML models, streamlining the end-to-end ML lifecycle in a unified environment. The complexities of data preparation, model development, bias detection, explainability, and monitoring are abstracted away behind an intuitive interface. Users don't need to be SageMaker AI or machine learning operations (MLOps) experts to develop, operationalize, and monitor models with SageMaker AI Canvas.

With SageMaker AI Canvas, the RAG functionality is provided through a no-code, document querying feature. You can enrich the chat experience in SageMaker AI Canvas by using an Amazon Kendra index as the underlying enterprise search. For more information, see [Extract information from documents with document querying](#).

Connecting SageMaker AI Canvas to the Amazon Kendra index requires a one-time setup. As part of the domain configuration, a cloud administrator can choose one or more Kendra indexes that the user can query when interacting with SageMaker Canvas. For instructions about how to enable the document querying feature, see [Getting started with using Amazon SageMaker AI Canvas](#).

SageMaker AI Canvas manages the underlying communication between Amazon Kendra and the selected foundation model. For more information about the foundation models that SageMaker AI Canvas supports, see [Generative AI foundation models in SageMaker AI Canvas](#). The following diagram shows how the document querying feature works after the cloud administrator has connected SageMaker AI Canvas to an Amazon Kendra index.



The diagram shows the following workflow:

1. The user starts a new chat in SageMaker AI Canvas, turns on **Query documents**, selects the target index, and then submits a question.
2. SageMaker AI Canvas uses the query to search the Amazon Kendra index for relevant data.
3. SageMaker AI Canvas retrieves the data and its sources from the Amazon Kendra index.
4. SageMaker AI Canvas updates the prompt to include the retrieved context from the Amazon Kendra index and submits the prompt to the foundation model.
5. The foundation model uses the original question and the retrieved context to generate an answer.
6. SageMaker AI Canvas provides the generated answer to the user. It includes references to the data sources, such as documents, that were used to generate the response.

Custom Retrieval Augmented Generation architectures on AWS

The previous section describes how to use a fully managed AWS service for Retrieval Augmented Generation (RAG). However, some use cases require more control over the system components, such as the retriever or the LLM (also called the *generator*). For example, you might need the flexibility to choose your own vector database or access an unsupported data source. For these use cases, you can build a custom RAG architecture.

This section contains the following topics:

- [Retrievers for RAG workflows](#)
- [Generators for RAG workflows](#)

For more information about how to choose between the retriever and generator options in this section, see [Choosing a RAG option](#) in this guide.

Retrievers for RAG workflows

This section explains how to build a retriever. You can use a fully managed semantic search solution, such as Amazon Kendra, or you can build a custom semantic search by using an AWS vector database.

Before you review the retriever options, make sure that you understand the three steps of the vector search process:

1. You separate the documents that need to be indexed into smaller parts. This is called *chunking*.
2. You use a process called [embedding](#) to convert each chunk into a mathematical vector. Then, you index each vector in a vector database. The approach that you use to index the documents influences the speed and accuracy of the search. The indexing approach depends on the vector database and the configuration options that it provides.
3. You convert the user query into a vector by using the same process. The retriever searches the vector database for vectors that are similar to the user's query vector. [Similarity](#) is calculated by using metrics such as Euclidean distance, cosine distance, or dot product.

This guide describes how to use the following AWS services or third-party services to build a custom retrieval layer on AWS:

- [Amazon Kendra](#)
- [Amazon OpenSearch Service](#)
- [Amazon Aurora PostgreSQL and pgvector](#)
- [Amazon Neptune Analytics](#)
- [Amazon MemoryDB](#)
- [Amazon DocumentDB](#)
- [Pinecone](#)
- [MongoDB Atlas](#)
- [Weaviate](#)

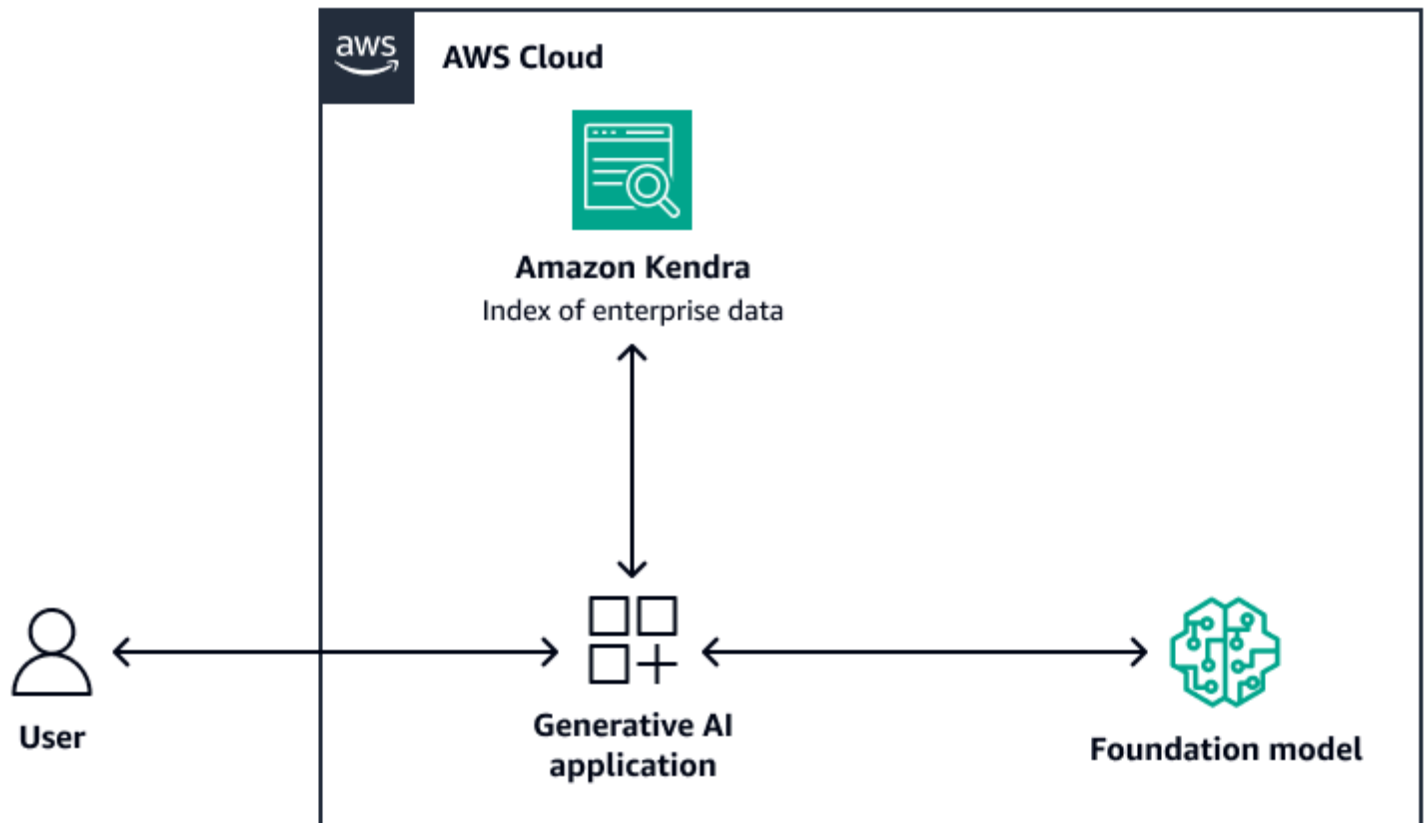
Amazon Kendra

[Amazon Kendra](#) is a fully managed, intelligent search service that uses natural language processing and advanced machine learning algorithms to return specific answers to search questions from your data. Amazon Kendra helps you directly ingest documents from multiple sources and query the documents after they have synced successfully. The syncing process creates the necessary infrastructure required to create a vector search on the ingested document. Therefore, Amazon Kendra does not require the traditional three steps of the vector search process. After the initial sync, you can use a defined schedule to handle ongoing ingestion.

The following are the advantages of using Amazon Kendra for RAG:

- You do not have to maintain a vector database because Amazon Kendra handles the entire vector search process.
- Amazon Kendra contains pre-built connectors for popular data sources, such as databases, website crawlers, Amazon S3 buckets, Microsoft SharePoint instances, and Atlassian Confluence instances. Connectors developed by AWS Partners are available, such as connectors for Box and GitLab.
- Amazon Kendra provides access control list (ACL) filtering that returns only documents that the end user has access to.
- Amazon Kendra can boost responses based on metadata, such as date or source repository.

The following image shows a sample architecture that uses Amazon Kendra as the retrieval layer of the RAG system. For more information, see [Quickly build high-accuracy Generative AI applications on enterprise data using Amazon Kendra, LangChain, and large language models](#) (AWS blog post).



For the foundation model, you can use Amazon Bedrock or an LLM deployed through [Amazon SageMaker AI JumpStart](#). You can use AWS Lambda with [LangChain](#) to orchestrate the flow between the user, Amazon Kendra, and the LLM. To build a RAG system that uses Amazon Kendra, LangChain, and various LLMs, see the [Amazon Kendra LangChain Extensions](#) GitHub repository.

Amazon OpenSearch Service

[Amazon OpenSearch Service](#) provides built-in ML algorithms for [k-nearest neighbors \(k-NN\) search](#) in order to perform a vector search. OpenSearch Service also provides a [vector engine for Amazon EMR Serverless](#). You can use this vector engine to build a RAG system that has scalable and high-performing vector storage and search capabilities. For more information about how to build a RAG system by using OpenSearch Serverless, see [Build scalable and serverless RAG workflows with a vector engine for Amazon OpenSearch Serverless and Amazon Bedrock Claude models](#) (AWS blog post).

The following are the advantages of using OpenSearch Service for vector search:

- It provides complete control over the vector database, including building a scalable vector search by using OpenSearch Serverless.
- It provides control over the chunking strategy.
- It uses approximate nearest neighbor (ANN) algorithms from the [Non-Metric Space Library \(NMSLIB\)](#), [Faiss](#), and [Apache Lucene](#) libraries to power a k-NN search. You can change the algorithm based on the use case. For more information about the options for customizing vector search through OpenSearch Service, see [Amazon OpenSearch Service vector database capabilities explained](#) (AWS blog post).
- OpenSearch Serverless integrates with Amazon Bedrock knowledge bases as a vector index.

Amazon Aurora PostgreSQL and pgvector

[Amazon Aurora PostgreSQL-Compatible Edition](#) is a fully managed relational database engine that helps you set up, operate, and scale PostgreSQL deployments. [pgvector](#) is an open-source PostgreSQL extension that provides vector similarity search capabilities. This extension is available for both Aurora PostgreSQL-Compatible and for Amazon Relational Database Service (Amazon RDS) for PostgreSQL. For more information about how to build a RAG-based system that uses Aurora PostgreSQL-Compatible and pgvector, see the following AWS blog posts:

- [Building AI-powered search in PostgreSQL using Amazon SageMaker AI and pgvector](#)
- [Leverage pgvector and Amazon Aurora PostgreSQL for Natural Language Processing, Chatbots, and Sentiment Analysis](#)

The following are the advantages of using pgvector and Aurora PostgreSQL-Compatible:

- It supports exact and approximate nearest neighbor search. It also supports the following similarity metrics: L2 distance, inner product, and cosine distance.
- It supports [Inverted File with Flat Compression \(IVFFlat\)](#) and [Hierarchical Navigable Small Worlds \(HNSW\)](#) indexing.
- You can combine the vector search with queries over domain-specific data that is available in the same PostgreSQL instance.
- Aurora PostgreSQL-Compatible is optimized for I/O and provides tiered caching. For workloads that exceed the available instance memory, pgvector can increase the queries per second for vector search by [up to 8 times](#).

Amazon Neptune Analytics

[Amazon Neptune Analytics](#) is a memory-optimized graph database engine for analytics. It supports a library of optimized graph analytic algorithms, low-latency graph queries, and vector search capabilities within graph traversals. It also has built-in vector similarity search. It provides one endpoint to create a graph, load data, invoke queries, and perform vector similarity search. For more information about how to build a RAG-based system that uses Neptune Analytics, see [Using knowledge graphs to build GraphRAG applications with Amazon Bedrock and Amazon Neptune](#) (AWS blog post).

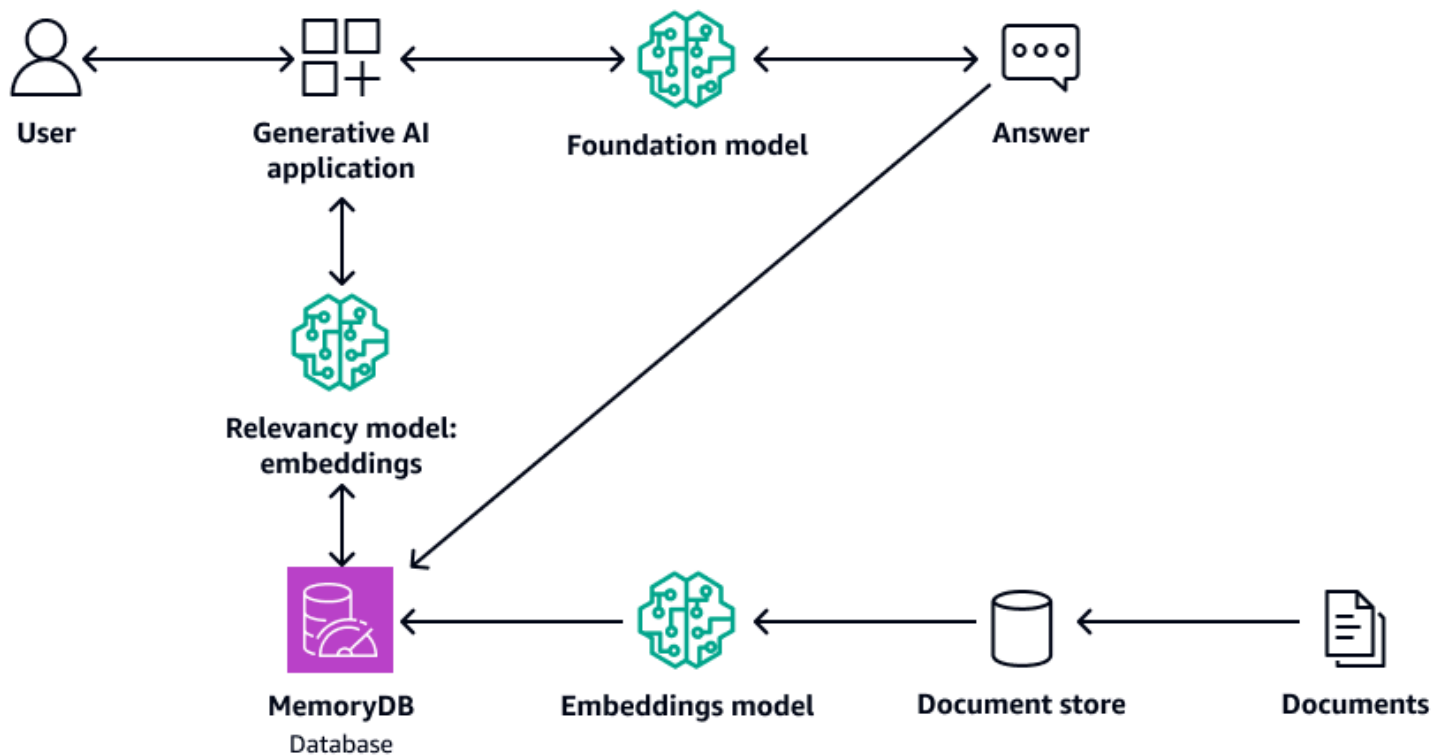
The following are the advantages of using Neptune Analytics:

- You can store and search embeddings in graph queries.
- If you integrate Neptune Analytics with LangChain, this architecture supports natural language graph queries.
- This architecture stores large graph datasets in memory.

Amazon MemoryDB

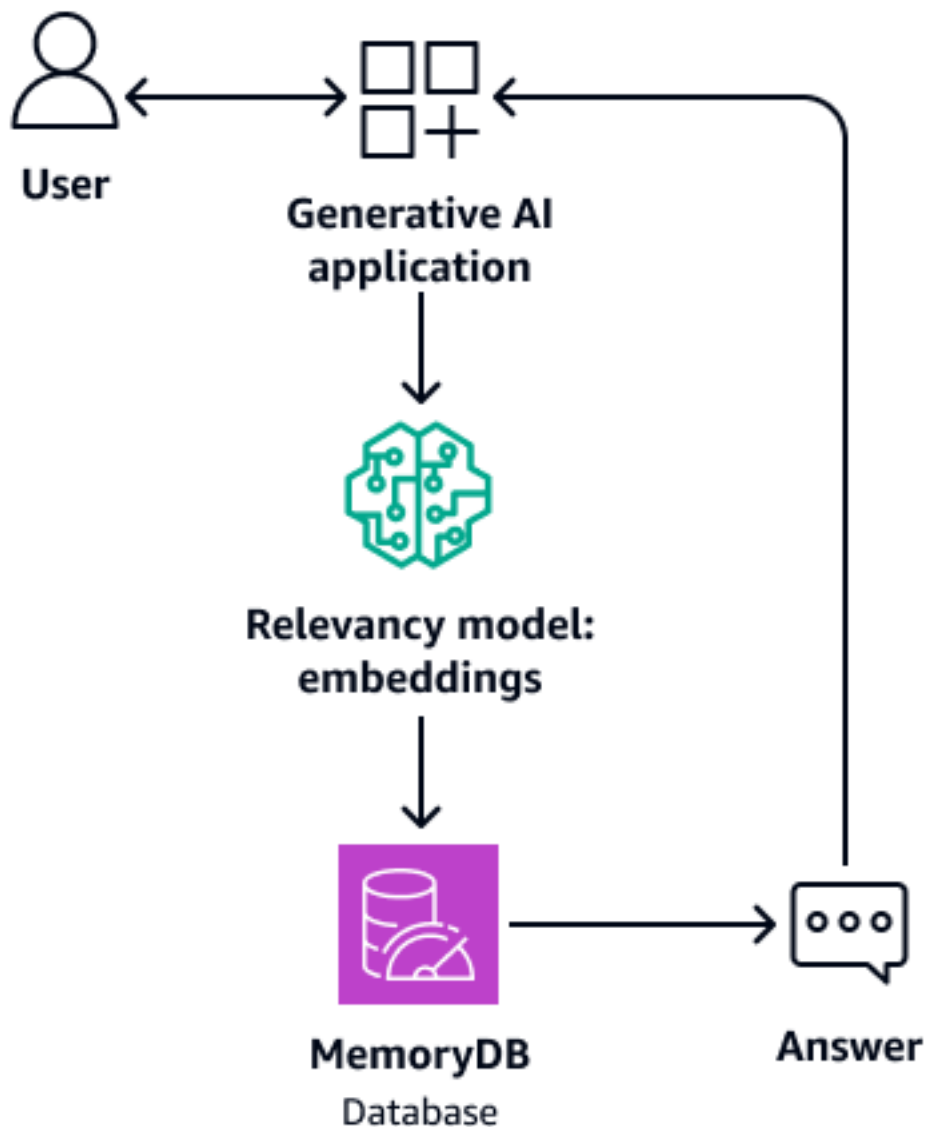
[Amazon MemoryDB](#) is a durable, in-memory database service that delivers ultra-fast performance. All of your data is stored in memory, which supports microsecond read, single-digit millisecond write latency, and high throughput. [Vector search for MemoryDB](#) extends the functionality of MemoryDB and can be used in conjunction with existing MemoryDB functionality. For more information, see the [Question answering with LLM and RAG](#) repository on GitHub.

The following diagram shows a sample architecture that uses MemoryDB as the vector database.



The following are the advantages of using MemoryDB:

- It supports both Flat and HNSW indexing algorithms. For more information, see [Vector search for Amazon MemoryDB is now generally available](#) on the AWS News Blog
- It can also act as a buffer memory for the foundation model. This means that previously answered questions are retrieved from the buffer instead of going through the retrieval and generation process again. The following diagram shows this process.



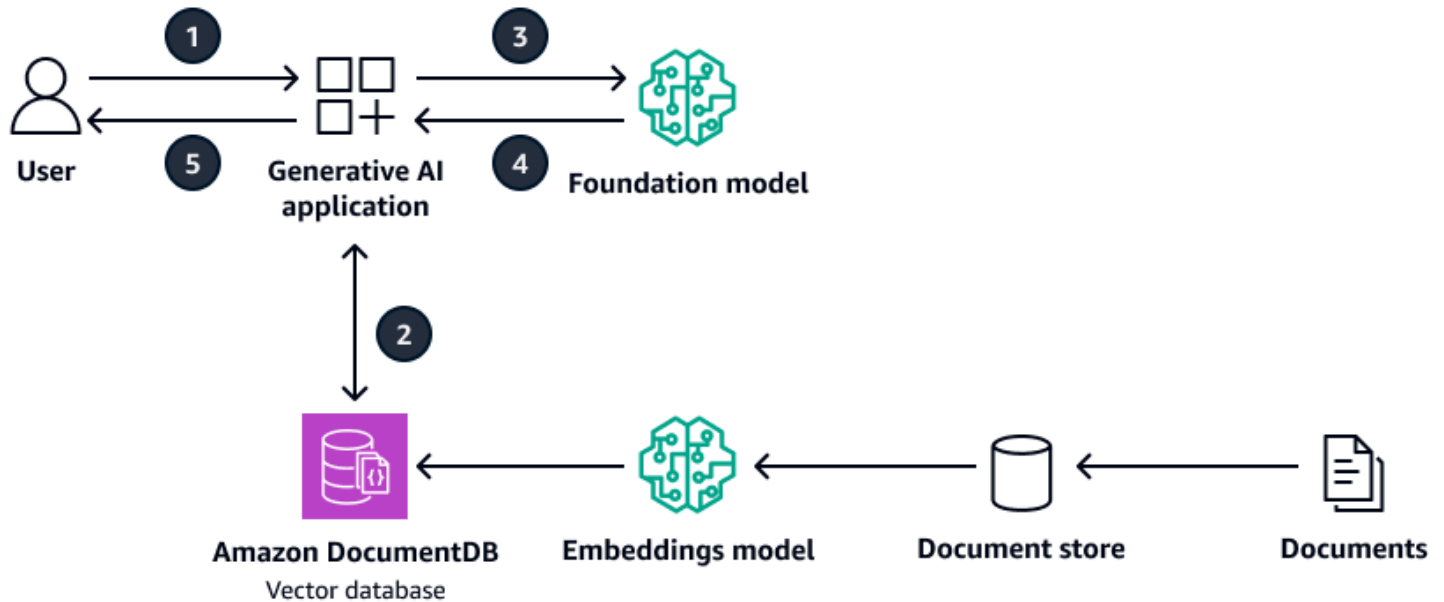
- Because it uses an in-memory database, this architecture provides single-digit millisecond query time for the semantic search.
- It provides up to 33,000 queries per second at 95-99% recall and 26,500 queries per second at greater than 99% recall. For more information, see the [AWS re:Invent 2023 - Ultra-low latency vector search for Amazon MemoryDB](#) video on YouTube.

Amazon DocumentDB

[Amazon DocumentDB \(with MongoDB compatibility\)](#) is a fast, reliable, and fully managed database service. It makes it easy to set up, operate, and scale MongoDB-compatible databases in the cloud. [Vector search for Amazon DocumentDB](#) combines the flexibility and rich querying capability of a

JSON-based document database with the power of vector search. For more information, see the [Question answering with LLM and RAG](#) repository on GitHub.

The following diagram shows a sample architecture that uses Amazon DocumentDB as the vector database.



The diagram shows the following workflow:

1. The user submits a query to the generative AI application.
2. The generative AI application performs a similarity search in the Amazon DocumentDB vector database and retrieves the relevant document extracts.
3. The generative AI application updates the user query with the retrieved context and submits the prompt to the target foundation model.
4. The foundation model uses the context to generate a response to the user's question and returns the response.
5. The generative AI application returns the response to the user.

The following are the advantages of using Amazon DocumentDB:

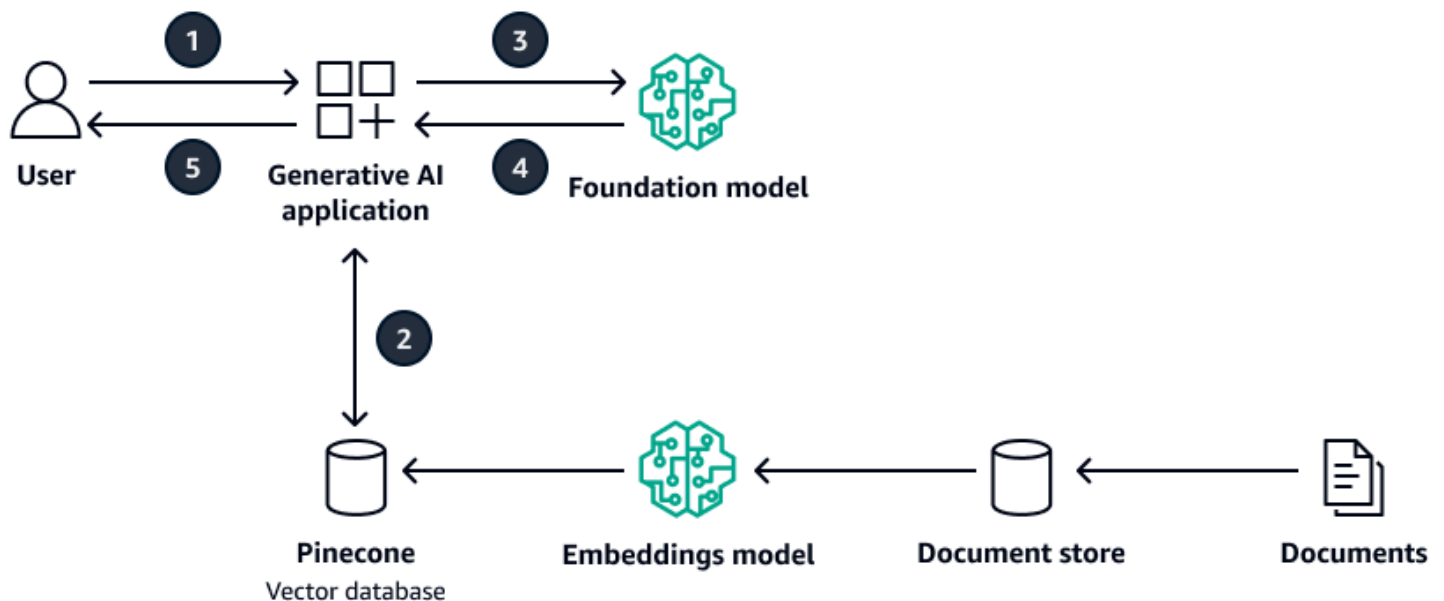
- It supports both HNSW and IVFFlat indexing methods.
- It supports up to 2,000 dimensions in the vector data and supports the Euclidean, cosine, and dot product distance metrics.
- It provides millisecond response times.

Pinecone

[Pinecone](#) is a fully managed vector database that helps you add vector search to production applications. It is available through the [AWS Marketplace](#). Billing is based on usage, and charges are calculated by multiplying the pod price by the pod count. For more information about how to build a RAG-based system that uses Pinecone, see the following AWS blog posts:

- [Mitigate hallucinations through RAG using Pinecone vector database & Llama-2 from Amazon SageMaker AI JumpStart](#)
- [Use Amazon SageMaker AI Studio to build a RAG question answering solution with Llama 2, LangChain, and Pinecone for fast experimentation](#)

The following diagram shows a sample architecture that uses Pinecone as the vector database.



The diagram shows the following workflow:

1. The user submits a query to the generative AI application.
2. The generative AI application performs a similarity search in the Pinecone vector database and retrieves the relevant document extracts.
3. The generative AI application updates the user query with the retrieved context and submits the prompt to the target foundation model.
4. The foundation model uses the context to generate a response to the user's question and returns the response.

5. The generative AI application returns the response to the user.

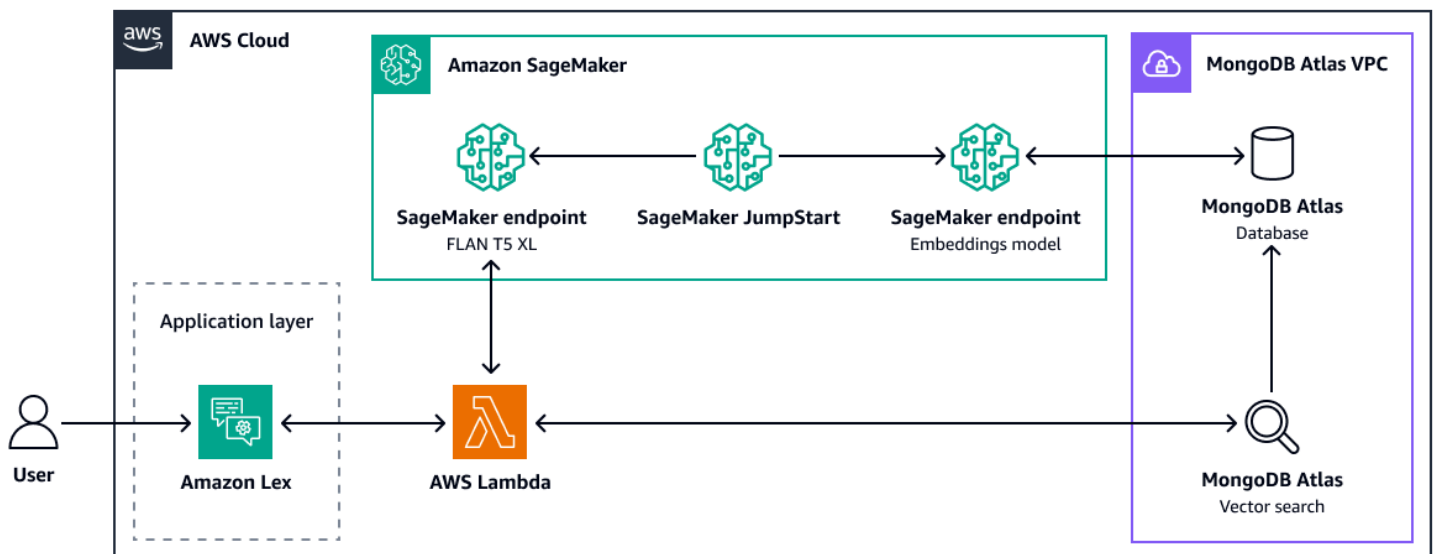
The following are the advantages of using Pinecone:

- It's a fully managed vector database and takes away the overhead of managing your own infrastructure.
- It provides the additional features of filtering, live index updates, and keyword boosting (hybrid search).

MongoDB Atlas

[MongoDB Atlas](#) is a fully managed cloud database that handles all the complexity of deploying and managing your deployments on AWS. You can use [Vector search for MongoDB Atlas](#) to store vector embeddings in your MongoDB database. Amazon Bedrock knowledge bases supports MongoDB Atlas for vector storage. For more information, see [Get Started with the Amazon Bedrock Knowledge Base Integration](#) in the MongoDB documentation.

For more information about how to use MongoDB Atlas vector search for RAG, see [Retrieval-Augmented Generation with LangChain, Amazon SageMaker AI JumpStart, and MongoDB Atlas Semantic Search](#) (AWS blog post). The following diagram shows the solution architecture detailed in this blog post.



The following are the advantages of using MongoDB Atlas vector search:

- You can use your existing implementation of MongoDB Atlas to store and search vector embeddings.
- You can use the [MongoDB Query API](#) to query the vector embeddings.
- You can independently scale the vector search and database.
- Vector embeddings are stored near the source data (documents), which improves the indexing performance.

Weaviate

[Weaviate](#) is a popular open source, low-latency vector database that supports multimodal media types, such as text and images. The database stores both objects and vectors, which combines vector search with structured filtering. For more information about using Weaviate and Amazon Bedrock to build a RAG workflow, see [Build enterprise-ready generative AI solutions with Cohere foundation models in Amazon Bedrock and Weaviate vector database on AWS Marketplace](#) (AWS blog post).

The following are the advantages of using Weaviate:

- It is open source and backed by a strong community.
- It is built for hybrid search (both vectors and keywords).
- You can deploy it on AWS as a managed software as a service (SaaS) offering or as a Kubernetes cluster.

Generators for RAG workflows

[Large language models \(LLMs\)](#) are very large [deep learning](#) models that are pretrained on vast amounts of data. They are incredibly flexible. LLMs can perform varied tasks, such as answering questions, summarizing documents, translating languages, and completing sentences. They have the potential to disrupt content creation and the way people use search engines and virtual assistants. While not perfect, LLMs demonstrate a remarkable ability to make predictions based on a relatively small prompt or number of inputs.

LLMs are a critical component of a RAG solution. For custom RAG architectures, there are two AWS services that serve as the primary options:

- [Amazon Bedrock](#) is a fully managed service that makes LLMs from leading AI companies and Amazon available for your use through a unified API.

- [Amazon SageMaker AI JumpStart](#) is an ML hub that offers foundation models, built-in algorithms, and prebuilt ML solutions. With SageMaker AI JumpStart, you can access pretrained models, including foundation models. You can also use your own data to fine-tune the pretrained models.

Amazon Bedrock

Amazon Bedrock offers industry-leading models from Anthropic, Stability AI, Meta, Cohere, AI21 Labs, Mistral AI, and Amazon. For a complete list, see [Supported foundation models in Amazon Bedrock](#). Amazon Bedrock also allows you to customize models with your own data.

You can [evaluate the model performance](#) to determine which are best suited for your RAG use case. You can test the latest models and also test to see which capabilities and features provide the best results and for the best price. The Anthropic Claude Sonnet model is a common choice for RAG applications because it excels at a wide range of tasks and provides a high degree of reliability and predictability.

SageMaker AI JumpStart

SageMaker AI JumpStart provides pretrained, open source models for a wide range of problem types. You can incrementally train and fine-tune these models before deployment. You can access the pretrained models, solution templates, and examples through the SageMaker AI JumpStart landing page in [Amazon SageMaker AI Studio](#) or use the [SageMaker AI Python SDK](#).

SageMaker AI JumpStart offers state-of-the-art foundation models for use cases such as content writing, code generation, question answering, copywriting, summarization, classification, information retrieval, and more. Use JumpStart foundation models to build your own generative AI solutions and integrate custom solutions with additional SageMaker AI features. For more information, see [Getting started with Amazon SageMaker AI JumpStart](#).

SageMaker AI JumpStart onboards and maintains publicly available foundation models for you to access, customize, and integrate into your ML life cycles. For more information, see [Publicly available foundation models](#). SageMaker AI JumpStart also includes proprietary foundation models from third-party providers. For more information, see [Proprietary foundation models](#).

Choosing a Retrieval Augmented Generation option on AWS

The [Fully managed RAG options](#) and [Custom RAG architectures](#) sections of this guide describe various approaches for building a RAG-based search solution on AWS. This section describes how to select between these options based on your use case. In some situations, more than one option might work. In that scenario, the choice depends on the ease of implementation, skills available in your organization, and your company's policies and standards.

We recommend that you consider the fully managed and custom RAG options in the following sequence and choose the first option that fits your use case:

1. Use [Amazon Q Business](#) unless:
 - This service is not available in your AWS Region, and your data cannot be moved to a Region where it is available
 - You have a specific reason to customize the RAG workflow
 - You want to use an existing vector database or a specific LLM
2. Use [knowledge bases for Amazon Bedrock](#) unless:
 - You have a vector database that is not supported
 - You have a specific reason to customize the RAG workflow
3. Combine [Amazon Kendra](#) with your choice of [generator](#) unless:
 - You want to choose your own vector database
 - You want to customize the chunking strategy
4. If you want more control over the retriever and want to select your own vector database:
 - If you don't have an existing vector database and don't need low latency or graph queries, consider using [Amazon OpenSearch Service](#).
 - If you have an existing PostgreSQL vector database, consider using the [Amazon Aurora PostgreSQL and pgvector](#) option.
 - If you need low latency, consider an in-memory option, such as [Amazon MemoryDB](#) or [Amazon DocumentDB](#).
 - If you want to combine vector search with a graph query, consider [Amazon Neptune Analytics](#).
 - If you are already using a third-party vector database or find a specific benefit from one, consider [Pinecone](#), [MongoDB Atlas](#), and [Weaviate](#).

5. If you want to choose an LLM:

- If you use Amazon Q Business, you can't choose the LLM.
- If you use Amazon Bedrock, you can choose one of the [supported foundation models](#).
- If you use Amazon Kendra or a custom vector database, you can use one of the [generators](#) described in this guide or use a custom LLM.

Note

You can also use your custom documents to fine-tune an existing LLM to increase the accuracy of its responses. For more information, see [Comparing RAG and fine-tuning](#) in this guide.

6. If you have an existing implementation of Amazon SageMaker AI Canvas that you want to use or if you want to compare RAG responses from different LLMs, consider [Amazon SageMaker AI Canvas](#).

Conclusion

This guide describes the various options for building a Retrieval Augmented Generation (RAG) system on AWS. You can start with fully managed services, such as Amazon Q Business and Amazon Bedrock knowledge bases. If you want more control over the RAG workflow, you can choose a custom retriever. For a generator, you can use an API to call a supported LLM in Amazon Bedrock, or you can deploy your own LLM by using Amazon SageMaker AI JumpStart. Review the recommendations in [Choosing a RAG option](#) to determine which option is best suited for your use case. After you select the best option for your use case, use the references provided in this guide to start building your RAG-based application.

Document history

The following table describes significant changes to this guide.

Change	Description	Date
Initial publication	—	October 28, 2024